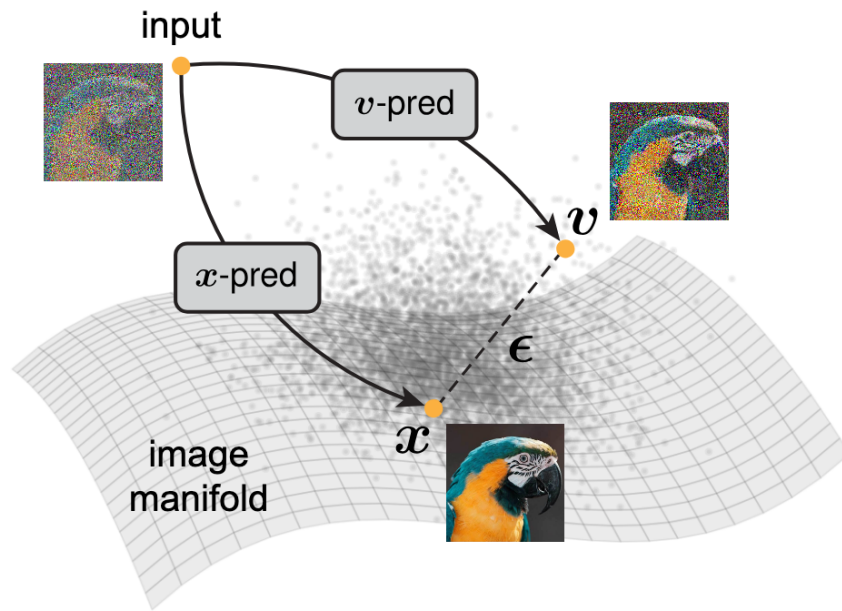


Back to Basics: Let Denoising Generative Models Denoise



Diffusion models should predict the image, rather than noise/score/velocities.

Recall: Diffusion Models and Flows.

In industrial applications (Stable Diffusion 3 / Flux), the flow-matching objective is mostly widely-used for its simplicity and numerical stability. In the forward process, a noisy sample $\mathbf{z}_t$ is drawn by a simple linear interpolation:

$$\mathbf{z}_t = t \mathbf{x} + (1 - t) \boldsymbol{\epsilon}$$

A flow-velocity $\mathbf{v}$ is the time-derivative of $\mathbf{z}$, that is

$$\mathbf{v} = \frac{d\mathbf{z}_t}{dt} = \mathbf{x} - \boldsymbol{\epsilon}.$$

The flow based methods minimize a loss function defined as

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}, \boldsymbol{\epsilon}} \|\mathbf{v}_\theta(\mathbf{z}_t, t) - \mathbf{v}\|^2,$$

where $\mathbf{v}_\theta$ is the direct output of a neural network.

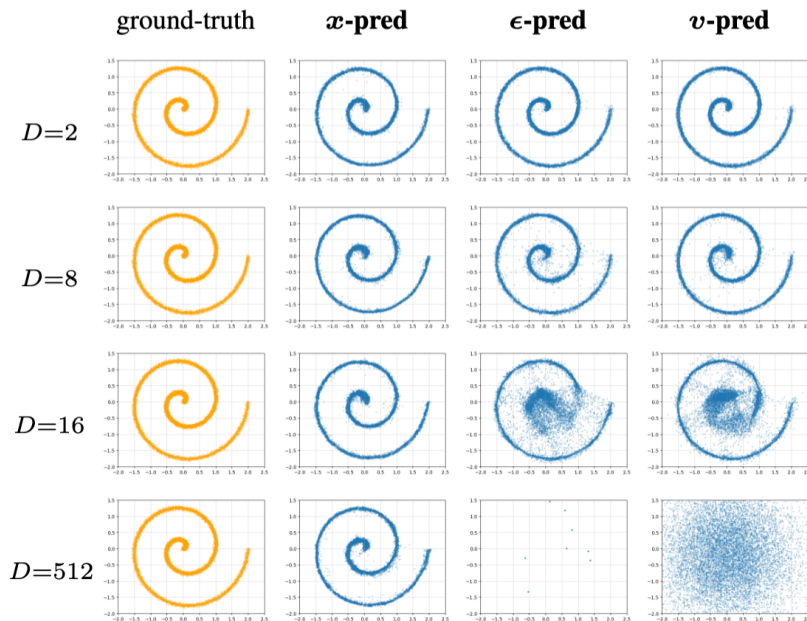
Reparameterizations of Diffusion Models: 3 Perspectives

	<u>x-pred</u> $\mathbf{x}_\theta := \text{net}_\theta(\mathbf{z}_t, t)$	ϵ -pred $\boldsymbol{\epsilon}_\theta := \text{net}_\theta(\mathbf{z}_t, t)$	$\mathbf{v}$ -pred $\mathbf{v}_\theta := \text{net}_\theta(\mathbf{z}_t, t)$
x -loss: $\mathbb{E} \ \mathbf{x}_\theta - \mathbf{x}\ ^2$	$\mathbf{x}_\theta$	$\mathbf{x}_\theta = (\mathbf{z}_t - (1-t)\boldsymbol{\epsilon}_\theta)/t$	$\mathbf{x}_\theta = (1-t)\mathbf{v}_\theta + \mathbf{z}_t$
ϵ -loss: $\mathbb{E} \ \boldsymbol{\epsilon}_\theta - \boldsymbol{\epsilon}\ ^2$	$\boldsymbol{\epsilon}_\theta = (\mathbf{z}_t - t\mathbf{x}_\theta)/(1-t)$	$\boldsymbol{\epsilon}_\theta$	$\boldsymbol{\epsilon}_\theta = \mathbf{z}_t - t\mathbf{v}_\theta$
$\mathbf{v}$ -loss: $\mathbb{E} \ \mathbf{v}_\theta - \mathbf{v}\ ^2$	$\mathbf{v}_\theta = (\mathbf{x}_\theta - \mathbf{z}_t)/(1-t)$	$\mathbf{v}_\theta = (\mathbf{z}_t - \boldsymbol{\epsilon}_\theta)/t$	$\mathbf{v}_\theta$

- The choice of network predictions are mathematically equivalent.
- The choice of loss are mathematically equivalent up to some weights, e.g. under $\mathbf{v}$ -pred, $\mathbb{E} \|\boldsymbol{\epsilon}_\theta - \boldsymbol{\epsilon}\|^2 = \mathbb{E} t^2 \|\mathbf{v}_\theta - \mathbf{v}\|^2$.
- There has been works showing that the **$\mathbf{v}$ -loss is superior** in terms of numerical stability and sampling speed.
- The key question: **could the neural network learn equally well in different output spaces?**

Claim: We should use the network to predict $\mathbf{x}$ directly because of the **manifold assumption**.

Learning low-dimensional distributions in high-dimensional space



The data $\mathbf{x} \in \mathbb{R}^D$ always lies in a 2-dimensional subspace “buried” in a D -dimensional space. Simple generative models (5-layer ReLU MLP with 256-dim hidden units) are trained on the data.

$\mathbf{x}$ -prediction can work well even when the model is **under-complete**.

Experiments

	x -pred	ϵ -pred	v -pred
x -loss	10.14	379.21	107.55
ϵ -loss	10.45	394.58	126.88
v -loss	8.62	372.38	96.53

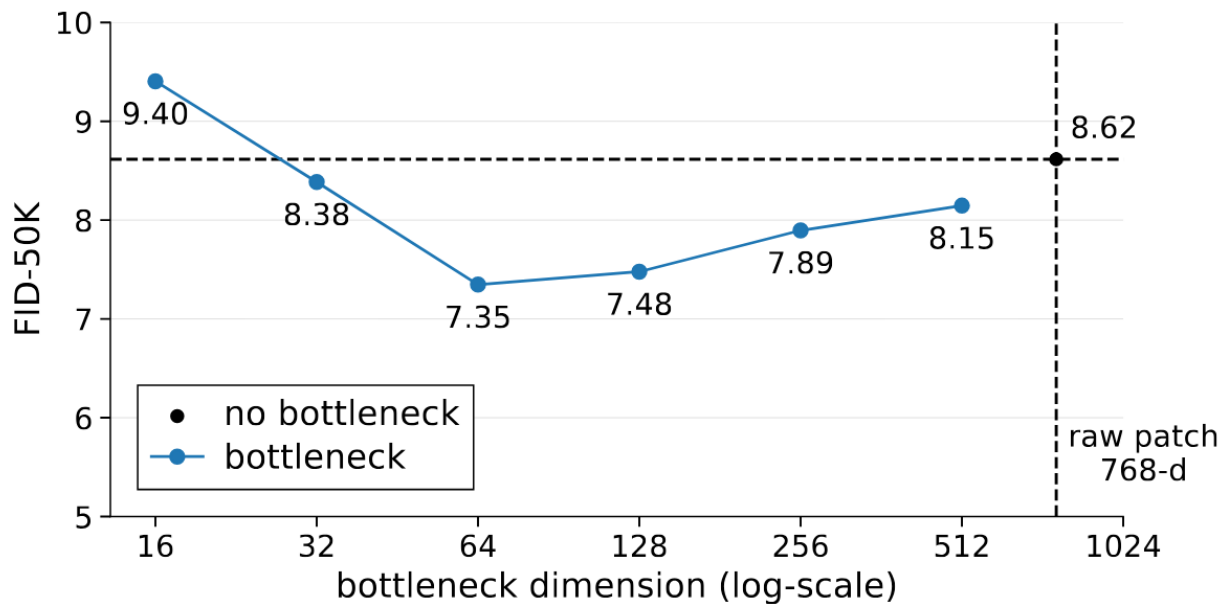
(a) ImageNet 256×256 , JiT-B/16

	x -pred	ϵ -pred	v -pred
x -loss	5.76	6.20	6.12
ϵ -loss	3.56	4.02	3.76
v -loss	3.55	3.63	3.46

(b) ImageNet 64×64 , JiT-B/4

- x -prediction is critical in high-dimensional settings.
- The choice of loss (weight) is not sufficient.

Experiments



Bottleneck can be beneficial: introducing low-dimensional intermediate linear layers in the model improves performance.

Model Architecture: “Just Image Transformers”

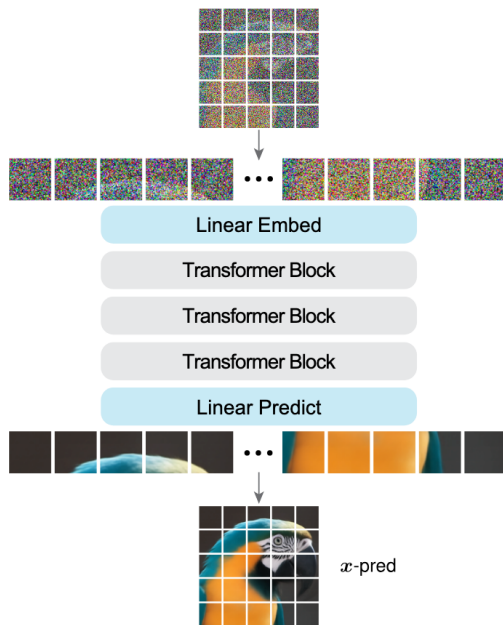


Figure 3. The “Just image Transformer” (JiT) architecture: simply a plain ViT [13] on patches of pixels for x -prediction.

Increasing Hidden Units is not necessary for larger images

resolution	model	len	patch dim	hid dens	params	Gflops	FID
256×256	JiT-B/16	256	768	768	131	25	4.37
512×512	JiT-B/32	256	3072	768	133	26	4.64
1024×1024	JiT-B/64	256	12288	768	141	30	4.82

Table. Model specifications and image quality (FID)

256 × 256	200 – ep	600 – ep	512 × 512	200 – ep	600 – ep
JiT-B/16	4.37	3.66	JiT-B/32	4.64	4.02
JiT-L/16	2.79	2.36	JiT-L/32	3.06	2.53
JiT-H/16	2.29	1.86	JiT-H/32	2.51	1.94
JiT-G/16	2.15	1.82	JiT-G/32	2.11	1.78

Table. FID of different model sizes/traning epochs. Table. FID of different model sizes/training epochs.